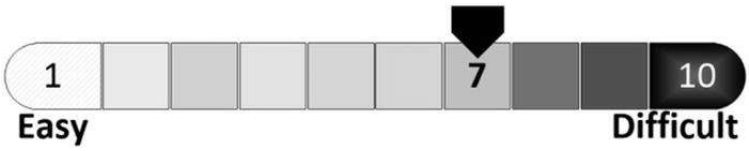
B. Choi, *Introduction to Python Network Automation*
https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-6806-3_13

13. Python Network Automation Lab: Basic Telnet

Brendan Choi[1]
(1) Sydney, NSW, Australia

This chapter is dedicated to Telnet labs using Python's `telnetlib` library. You will learn how to use a basic Telnet example to reiterate and convert your tasks performed on the keyboard into Python scripts. Although Telnet is an insecure protocol and the best practice is to use the SSH protocol for remote connections, it is still in use today in many production environments residing in a well-secured network where only a small number of engineers can access the environment. At the end of this lab, you will be able to do the following using the basic Telnet library: interact with network devices on Python interpreter session, configure a single and multiple-switch VLAN using a templated approach using loops, perform basic network engineer administration tasks such as adding users via an external file source, and make backup copies of `running-config` or `startup-configs` to the local Python server directory. In the next chapter, we will work with the Python SSH library to expand on the ideas learned in this chapter.



Python Network Automation: Telnet Labs

In this book, many hours were invested into making the lab experience close to a real physical equipment lab as possible so you can have a realistic lab experience without the hardware or worrying about different connector types. Under the current lab setup on your host PC, you can write code on three different operating systems: host PC on Windows 10, CentOS 8.1 Linux server, and Ubuntu 20.04 LTS Linux VM server. If you want to add more places to write code and test it, you can clone any of the Linux VMs. In Chapter 2, you were introduced to Python 3 on Windows OS, and then in Chapters 4, 5, and 6, you did most of the exercises on the Linux command-line interface (CLI). Companies almost never install Python on a Windows server and use it in a production environment. In other words, the vast majority of Python applications run on security-hardened Linux servers. Although Microsoft has introduced the concept of Windows Subsystem for Linux (WSL) and it can run Linux systems on

Windows and is extremely easy to use, the actual virtual machine running on Windows is Linux OS.

The current craze of network automation is for fewer engineers who can do more work using an application programming interface (API) without opening a CLI console, Telnet, or SSH terminal windows. The introduction of API support across many newer networking platforms has also drastically reduced graphical user interface (GUI) use in the networking industry. However, industry insiders say the traditional way of logging into network devices and managing them will remain in place for many more years to come. We still use our keyboard to write our Python code and also call the API libraries used in our code, but unless we are API developers, we probably do not fully understand the inner workings of each API. At the end of the day, network automation is the act of mimicking how an experienced network engineer thinks. A good example of such a product is Red Hat's Ansible, which relies on the agentless SSH connection, instead of an API. In this sense, the API is the reinterpretation of this but for machine-to-machine interaction without the use of the keyboard and screen. In this chapter, you will start writing Python Telnet scripts to interact with the network devices in our basic lab topology. Even though using an API is becoming the new norm for administering enterprise networking devices, for now and the foreseeable future, there will still be many devices without API interfaces, and you still need to connect to these devices remotely using either a Telnet or SSH connection. This means we can still use Python in network automation, relying on Telnet and SSH libraries. Anyway, the necessary skills in writing Python code do not change; only the access methods and libraries change. See Figure **13-1**.

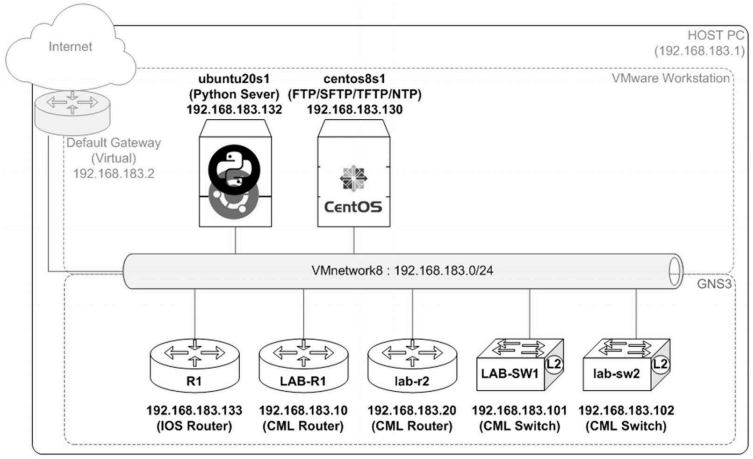


Figure 13-1. cmllab-basic, logical topology

In a highly secured client environment, you can only manage core network and system devices from designated servers called *jump hosts*. Jump hosts can be Windows or Linux OS based. The `cmllab-basic` GNS3 topology emulates this environment, and you have to use PuTTY on your host PC to SSH into the `ubuntu20s1` (Python and jump host) server to manage devices in the network. The `centos8s1` server provides IP services we will rely on for various lab scenarios. We will write Python scripts in Windows Notepad++ or enter the scripts directly on the `ubuntu20s1` server's text editor via an SSH session for the entire chapter. You can enter the Python script directly into the Ubuntu server for those who are fast and accurate typists. Still, if you prefer to work with the script from Windows Notepad or Notepad++ and cut and paste it to the Ubuntu server later, that is also an acceptable method. Let's fire up all devices in

the `cmllab-basic` project, write some Python code, and see how Telnet Python scripts interact with Cisco routers and switches.

Telnet Lab 1: Interactive Telnet Session to Cisco Devices on a Python Interpreter

For this lab, you will need to power on the `ubuntu20s1` Linux server (192.168.183.132), `LAB-SW1` (192.168.183.101), and `LAB-R1` (192.168.183.10). You will also power on the IOS router, `R1` (192.168.183.133), to check the OSPF neighborship after running the first script. If you are using another subnet to connect to your NAT (`VMnetwork8`), your IP addresses will be different from this book, and you will have to replace the IP addresses. See Figure **13-2**.

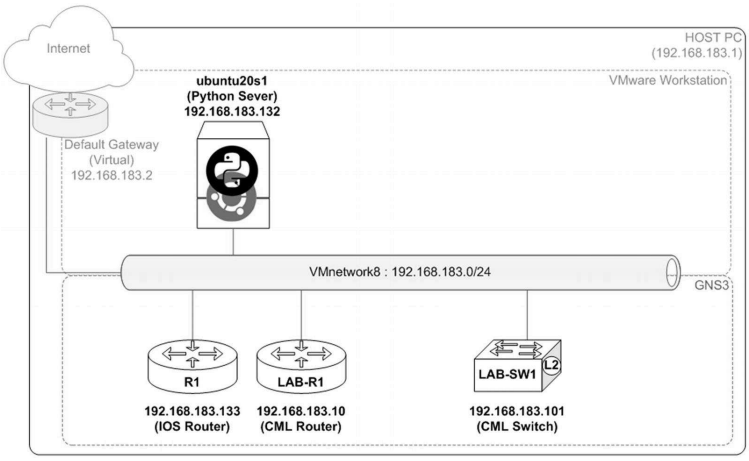


Figure 13-2. Telnet lab 1, devices in use

You will write a Telnet script and run it from the `ubuntu20s1` Python server to Telnet into the `LAB-R1` router to make configuration changes. At the end of this lab, `LAB-R1` must have the following configurations in the `running-config` file :

- Loopback 0 with IP address of 2.2.2.2/32
- Loopback 1 with IP address of 4.4.4.4/32
- OSPF 1 with network 0.0.0.0 255.255.255.255 in area 0

#	Task
1	Open PuTTY on your host PC and SSH into the `ubuntu20s1` (192.168.183.132) server. If you have assigned a different IP address, please use your server IP address. If you forgot about which IP address was assigned to the `ubuntu20s1` server, open the Linux console from VMware Workstation and log in; then run the `ip address` command .

Now log in with your username and password.

```
login as: pynetauto
```

```
pynetauto@192.168.183.132's password: *********
```

```
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-47-generic x86_64)
```

#	Task

```
* Documentation: https://help.ubuntu.com
```

```
* Management: https://landscape.canonical.com
```

```
* Support: https://ubuntu.com/advantage
```

```
System information disabled due to load higher than 1.0
```

```
327 updates can be installed immediately.
```

```
142 of these updates are security updates.
```

```
To see these additional updates run: apt list --upgradable
```

```
Last login: Fri Jan  8 23:26:01 2021 from 192.168.183.1
```

```
pynetauto@ubuntu20s1:~$
```

2 Currently, to run Python from `ubuntu20s1`, we have to issue the `python3` command. Since there is no Python 2.7 installed and Python 3 is the only version you are going to be using here, you can use an `alias` command to shorten `python3` to `python`. Follow the instruction here to use the alias in Linux :

```
pynetauto@ubuntu20s1:~$ users
```

```
pynetauto
```

```
pynetauto@ubuntu20s1:~$ pwd
```

```
/home/pynetauto
```

```
pynetauto@ubuntu20s1:~$ nano /home/pynetauto/.bashrc
```

Once you open the `.bashrc` file in the `/home/user/` directory, add a new line, `alias python=python3`, to the `.bashrc` file; this is similar to Figure **13-1**. Once you finish adding a line, press Ctrl+X to save and exit the file.

```
GNU nano 4.8                                    /home/pynetauto/.bashrc
```

```
# alias ADDED by pynetauto
```

```
alias python=python3
```

```
^G Get Help  ^O Write Out   ^W Where Is   ^K Cut Text     ^J Justify    ^C Cur
Pos
```

```
^X Exit      ^R Read File   ^\ Replace    ^U Paste Text   ^T To Spell   ^_ Go To
Line
```

#	Task
3	After adding the alias line, run the `source ~/.bashrc` command to restart the user's `bashrc`. You can now use the `python` or `python3` command to run Python 3.8.2 on your `ubuntu20s1` server .

```
pynetauto@ubuntu20s1:~$ source ~/.bashrc
```

```
pynetauto@ubuntu20s1:~$ python
```

```
Python 3.8.2 (default, Jul 16 2020, 14:00:26)
```

```
[GCC 9.3.0] on linux
```

```
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>>
```

4	As we have done in the IOS Telnet lab, go to https://docs.python.org/3/library/telnetlib.html and scroll down to the bottom of the page. Locate a sample Telnet code. You can alternatively reuse the old code from the `R1` IOS lab from Chapter **10**.8. Let's copy this code to make our first Telnet script. This code will be used throughout the Telnet labs, so here are quick explanations of what each line does in the given Python script. Since we are using Cisco devices, more explanations are provided concerning Cisco Telnet sessions .

Telnet Example

A simple example illustrates the typical use :

`import getpass`	# import getpass module for password
`import telnetlib`	# import telnetlib module for telnet
`HOST = "localhost"`	# IP address or hostname of device to connect
`user = input("Enter your remote account: ")`	# User input for UserID
`password = getpass.getpass()`	# define a variable password as getpass.getpass function class
`tn = telnetlib.Telnet(HOST)`	# define a variable tn as telnetlib.Telnet type class
`tn.read_until(b"login: ")`	# "login: " appears on the telnet window, Python waits until "login: " binary information is returned to the login screen. On Cisco devices, the expected re-

#	Task	
		turned binary information is “Username: ”, this needs to be updated in your script.
	`tn.write(user.encode('ascii') + b"\n")`	# Encode userID in ASCII and sent to telnet virtual terminal
	`if password:`	# if prompted for password
	`tn.read_until(b"Password: ")`	# Wait for password until entered
	`tn.write(password.encode('ascii') + b"\n")`	# Encode password in ASCII and send to telnet virtual terminal
	`tn.write(b"ls\n")`	# ‘ls’ is a Linux command list. ls is not used in Cisco devices. Can be removed or modified.
	`tn.write(b"exit\n")`	# Send ‘exit’ command to telnet virtual terminal
	`print(tn.read_all().decode('ascii'))`	# Read all commands run in this session, decode in ASCII and print out on the user's screen.

5 Make a directory called `telnet_labs`, change the working directory, and then create the first CML Telnet script using the `touch` command . The script name is `add_lo_ospf1.py`. Follow these steps:

```
pynetauto@ubuntu20s1:~$ pwd
/home/pynetauto
pynetauto@ubuntu20s1:~$ mkdir telnet_labs
pynetauto@ubuntu20s1:~$ cd telnet_labs
pynetauto@ubuntu20s1:~/telnet_labs$ pwd
/home/pynetauto/telnet_labs
pynetauto@ubuntu20s1:~/telnet_labs$ touch add_lo_ospf1.py
pynetauto@ubuntu20s1:~/telnet_labs$ ls
7.2.1_add_lo_ospf1.py
```

6 To speed up the lab, use the nano text editor on the Linux server. First, use `nano add_lo_ospf1.py` to open the blank Python script. Then, copy and paste the Telnet example by using the right-click menu. Finally, modify the sections highlighted in the following source code :

```
#    Task

     GNU nano 4.8                    add_lo_ospf1.py

     import getpass

     import telnetlib

     HOST = "192.168.183.10"

     user = input("Enter your username: ")

     password = getpass.getpass()

     tn = telnetlib.Telnet(HOST)

     tn.read_until(b"Username: ")

     tn.write(user.encode('ascii') + b"\n")

     if password:

       tn.read_until(b"Password: ")

       tn.write(password.encode('ascii') + b"\n")

     tn.write(b"enable\n")

     tn.write(b"cisco123\n")

     tn.write(b"conf t\n")

     tn.write(b"int loopback 0\n")

     tn.write(b"ip add 2.2.2.2 255.255.255.255\n")

     tn.write(b"int loopback 1\n")

     tn.write(b"ip add 4.4.4.4 255.255.255.255\n")

     tn.write(b"router ospf 1\n")

     tn.write(b"network 0.0.0.0 255.255.255.255 area 0\n")

     tn.write(b"end\n")

     tn.write(b"exit\n")

     print(tn.read_all().decode('ascii'))
```

```
#   Task

    ^G Get Help  ^O Write Out   ^W Where Is   ^K Cut Text     ^J Justify    ^C Cur
    Pos

    ^X Exit      ^R Read File   ^\ Replace    ^U Paste Text   ^T To Spell   ^_ Go To
    Line
```

7 Before running the previous script from the `ubuntu20s1` server, go to `LAB-R1` and enable `debug telnet` to monitor the Telnet activities from `LAB-R1`.

```
LAB-R1#debug telnet

Incoming Telnet debugging is on
```

8 On `ubuntu20s1`, run the script using the `python add_lo_ospf1.py` command. When prompted for the username and password, enter your router username and password. Once the script runs successfully, your screen should look similar to this :

```
pynetauto@ubuntu20s1:~/telnet_labs$ python add_lo_ospf1.py

Enter your username: pynetauto

Password: **********

**************************************************************************

* IOSv is strictly limited to use for evaluation, demonstration and IOS  *

* education. IOSv is provided as-is and is not supported by Cisco's      *

* Technical Advisory Center. Any use or disclosure, in whole or in part, *

* of the IOSv Software or Documentation to any third party for any       *

* purposes is expressly prohibited except as otherwise authorized by     *

* Cisco in writing.                                                      *

**************************************************************************

LAB-R1#enable

LAB-R1#cisco123

Translating "cisco123"...domain server (192.168.183.2)

(192.168.183.2)

Translating "cisco123"...domain server (192.168.183.2)
```

```
#    Task
```

```
% Bad IP address or host name
```

```
% Unknown command or computer name, or unable to find computer address
```

```
LAB-R1#conf t
```

```
Enter configuration commands, one per line.  End with CNTL/Z.
```

```
LAB-R1(config)#int loopback 0
```

```
LAB-R1(config-if)#ip add 2.2.2.2 255.255.255.255
```

```
LAB-R1(config-if)#int loopback 1
```

```
LAB-R1(config-if)#ip add 4.4.4.4 255.255.255.255
```

```
LAB-R1(config-if)#router ospf 1
```

```
LAB-R1(config-router)#network 0.0.0.0 255.255.255.255 area 0
```

```
LAB-R1(config-router)#end
```

```
LAB-R1#exit
```

9 Go to `LAB-R1` and now check the console. You can see the Telnet activities that have just taken place .

```
LAB-R1#
```

```
*Jan 11 18:11:08.885: Telnet578: 1 1 251 1
```

```
*Jan 11 18:11:08.887: TCP578: Telnet sent WILL ECHO (1)
```

```
[...omitted for brevity]
```

```
*Jan 11 18:11:09.181: TCP578: Telnet received WONT TTY-TYPE (24)
```

```
*Jan 11 18:11:09.183: TCP578: Telnet received WONT WINDOW-SIZE (31)
```

```
LAB-R1#
```

```
*Jan 11 18:11:11.730: %LINEPROTO-5-UPDOWN: Line protocol on Interface Loopback0,
changed state to up
```

```
LAB-R1#
```

```
*Jan 11 18:11:12.972: %LINEPROTO-5-UPDOWN: Line protocol on Interface Loopback1,
changed state to up
```

#	Task

```
*Jan 11 18:11:13.791: %SYS-5-CONFIG_I: Configured from console by pynetauto on
vty0 (192.168.183.132)
```

```
LAB-R1#
```

10 From the `LAB-R1` console, run the `show ip interface brief` command to check the newly configured `Loopback0` and `Loopback1` configurations .

```
LAB-R1#show ip interface brief
```

```
Interface                  IP-Address      OK? Method
Status                Protocol
```

```
GigabitEthernet0/0         192.168.183.10  YES manual up                    up
```

```
GigabitEthernet0/1         172.168.1.1     YES manual up                    up
```

```
GigabitEthernet0/2         unassigned      YES unset  administratively down down
```

```
GigabitEthernet0/3         unassigned      YES unset  administratively down down
```

```
Loopback0                  2.2.2.2         YES manual up                    up
```

```
Loopback1                  4.4.4.4         YES manual up                    up
```

11 On the IOS router, `R1`, let's power on `R1` and observe if the OSPF neighborship forms correctly between `R1` and `LAB-R1` routers. Once the OSPF status changes from `LOADING` to `FULL, Loading Done`, run the `show ip ospf neighbor` command to check the neighborship. At this stage, you should be able to ping `LAB-R1`'s `Loopback0` (2.2.2.2) and `Loopback1` (4.4.4.4) interfaces from `R1`.

```
R1#
```

```
*Mar  1 00:21:25.807: %OSPF-5-ADJCHG: Process 1, Nbr 4.4.4.4 on FastEthernet0/0
from LOADING to FULL, Loading Done
```

```
R1#show ip ospf neighbor
```

```
Neighbor ID     Pri   State           Dead Time   Address         Interface
```

```
4.4.4.4           1   FULL/DR         00:00:39    192.168.183.10  FastEthernet0/0
```

```
R1#ping 2.2.2.2
```

```
Type escape sequence to abort.
```

```
Sending 5, 100-byte ICMP Echos to 2.2.2.2, timeout is 2 seconds:
```

#	Task

```
!!!!!

Success rate is 100 percent (5/5), round-trip min/avg/max = 8/11/20 ms

R1#ping 4.4.4.4

Type escape sequence to abort.

Sending 5, 100-byte ICMP Echos to 4.4.4.4, timeout is 2 seconds:

!!!!!

Success rate is 100 percent (5/5), round-trip min/avg/max = 8/13/28 ms
```

12	On `LAB-R1`, you will observe that the OSPF status changes from `LOADING` to `FULL, Loading Done`. Run the `show ip ospf neighbor` command to check the neighborship with `R1`. At this stage, you should be able to ping R1's f0/1 interface (7.7.7.2). Run the `undebug all` command to turn off Telnet debugging and complete your first CML-PERSONAL Telnet lab.

```
LAB-R1#

*Jan 11 18:11:21.417: %OSPF-5-ADJCHG: Process 1, Nbr 192.168.183.133 on
GigabitEthernet0/0 from LOADING to FULL, Loading Done

LAB-R1#show ip ospf neighbor

Neighbor ID     Pri   State           Dead Time   Address         Interface

192.168.183.133   1   FULL/BDR        00:00:39    192.168.183.133
GigabitEthernet0/0

LAB-R1#ping 7.7.7.2

Type escape sequence to abort.

Sending 5, 100-byte ICMP Echos to 7.7.7.2, timeout is 2 seconds:

!!!!!

Success rate is 100 percent (5/5), round-trip min/avg/max = 7/10/14 ms

LAB-R1#undebug all

All possible debugging has been turned off
```

All the Python code used can be downloaded from **https://github.com/pynetauto/apress_pynetauto**. If you enjoyed your first Telnet lab and want to try the second lab now, let's continue .

Telnet Lab 2: Configure a Single Switch with a Python Telnet Template

In this lab, you will be configuring some VLANs on `LAB-SW1` via Telnet. You will need to power on the `ubuntu20s1` Linux server (192.168.183.132) and `LAB-SW1` (192.168.183.101) for this lab. To save time, we will be copying the first script, `add_lo_ospf1.py`, and renaming it as `add_vlans_single.py`. At the end of this lab, `LAB-SW1` should be configured as shown in Figure **13-3**.

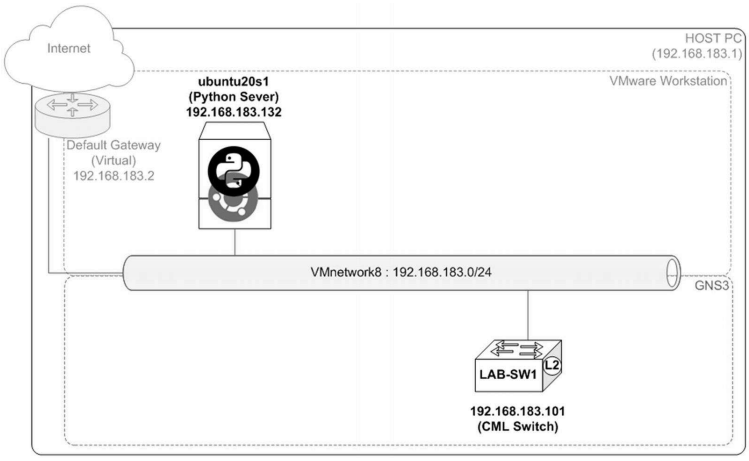


Figure 13-3. Telnet lab 2, devices in use

- Configure VLANs 2 to 5 with the following VLAN descriptions:
  - VLAN 2, `Data_vlan_2`
  - VLAN 3, `Data_vlan_3`
  - VLAN 4, `Voice_vlan_4`
  - VLAN 5, `Wireless_vlan_5`
- Configure GigabitEthernet 1/0 to GigabitEthernet 1/3 range as access switch ports with `Wireless_vlan_5`.
- Configure GigabitEthernet 2/0 to GigabitEthernet 2/3 range as switch ports with `Data_vlan_2` (Data VLAN) and `voice_vlan_4` (Auxiliary VLAN).
- Bring up all configured interfaces with the `no shut` command.

#	Task
1	SSH into the `ubuntu20s1` server (192.168.183.132) and continue working in the same directory as the first lab. Follow the instructions here to copy the first Python script and rename it as `add_vlans_single.py`.

```
pynetauto@ubuntu20s1:~$ cd telnet_labs
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ ls
```

```
7.2.1_add_lo_ospf1.py
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ cp add_lo_ospf1.py add_vlans_single.py
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ ls
```

```
#    Task
```

```
     add_lo_ospf1.py  add_vlans_single.py
```

2 Open the newly created `.py` file with a nano text editor. Press Ctrl+K to copy/cut lines of codes and press Ctrl+U to paste information. Note that the IP address of `LAB-SW1` is 192.168.183.101. Make sure you update the IP address next to `HOST` with this IP address.

Use the following information to modify your script. When writing code, every comma and full stop counts.

`b` = Byte literals; produces an instance of the `byte` type instead of the `str` type.

`\n` = Newline character to go to the next line. In other words, `"[Enter] key"encode('ascii')` means to use ASCII encoding for the switch/router.

```
     GNU nano 4.8                        add_vlans_single.py
```

```
     import getpass
```

```
     import telnetlib
```

```
     HOST = "192.168.183.101"
```

```
     user = input("Enter your username: ")
```

```
     password = getpass.getpass()
```

```
     tn = telnetlib.Telnet(HOST)
```

```
     tn.read_until(b"Username: ")
```

```
     tn.write(user.encode('ascii') + b"\n")
```

```
     if password:
```

```
        tn.read_until(b"Password: ")
```

```
        tn.write(password.encode('ascii') + b"\n")
```

```
     # Get into config mode
```

```
     tn.write(b"conf t\n")
```

```
     # configure 4 VLANs with VLAN names
```

```
     tn.write(b"vlan 2\n")
```

```
     tn.write(b"name Data_vlan_2\n")
```

```
#    Task
```

```
    tn.write(b"vlan 3\n")
```

```
    tn.write(b"name Data_vlan_3\n")
```

```
    tn.write(b"vlan 4\n")
```

```
    tn.write(b"name Voice_vlan_4\n")
```

```
    tn.write(b"vlan 5\n")
```

```
    tn.write(b"name Wireless_vlan_5\n")
```

```
    tn.write(b"exit\n")
```

```
    # configure Gi1/0 - Gi1/3 as access siwtchports and assign vlan 5 for
    wireless APs
```

```
    tn.write(b"interface range gi1/0 - 3\n")
```

```
    tn.write(b"switchport mode access\n")
```

```
    tn.write(b"switchport access vlan 5\n")
```

```
    tn.write(b"no shut\n")
```

```
    #configure gi2/0 - gi2/3 as access switchports and assign vlan 2 for data
    and vlan 4 for voice
```

```
    tn.write(b"interface range gi2/0 - 3\n")
```

```
    tn.write(b"switchport mode access \n")
```

```
    tn.write(b"switchport access vlan 2\n")
```

```
    tn.write(b"switchport voice vlan 4\n")
```

```
    tn.write(b"no shut\n")
```

```
    tn.write(b"end\n")
```

```
    tn.write(b"exit\n")
```

```
    print(tn.read_all().decode('ascii'))
```

```
    ^G Get Help  ^O Write Out   ^W Where Is   ^K Cut Text     ^J Justify    ^C
    Cur Pos
```

```
# Task

^X Exit      ^R Read File   ^\ Replace    ^U Paste Text   ^T To Spell   ^_
Go To Line
```

3 From the `ubuntu20s1` server, run the `ping 192.168.183.101 -c 4` command to check the connectivity.

```
pynetauto@ubuntu20s1:~/telnet_labs$ ping 192.168.183.101 -c 3
PING 192.168.183.101 (192.168.183.101) 56(84) bytes of data.
64 bytes from 192.168.183.101: icmp_seq=1 ttl=255 time=20.3 ms
64 bytes from 192.168.183.101: icmp_seq=2 ttl=255 time=7.77 ms
64 bytes from 192.168.183.101: icmp_seq=3 ttl=255 time=17.5 ms
--- 192.168.183.101 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 7.773/14.035/20.312/5.067 ms
```

4 Optionally, on `LAB-SW1`, enable `debug telnet` to capture Telnet activities during the script run. After debugging has been switched on, leave the console window open.

```
LAB-SW1#debug telnet
```

5 Now we checked the connectivity. Let's run the `python add_vlans_single.py` command to run the script, add new VLANs, and configure the designated switch ports.

```
pynetauto@ubuntu20s1:~/telnet_labs$ python add_vlans_single.py
Enter your username: pynetauto
Password:********
**************************************************************************
* IOSv is strictly limited to use for evaluation, demonstration and IOS  *
* education. IOSv is provided as-is and is not supported by Cisco's      *
* Technical Advisory Center. Any use or disclosure, in whole or in part, *
* of the IOSv Software or Documentation to any third party for any       *
* purposes is expressly prohibited except as otherwise authorized by     *
```

```
# Task

    * Cisco in writing.                                                  *

    ************************************************************************

    LAB-SW1#conf t

    Enter configuration commands, one per line.  End with CNTL/Z.

    LAB-SW1(config)#vlan 2

    LAB-SW1(config-vlan)#name Data_vlan_2

    LAB-SW1(config-vlan)#vlan 3

    LAB-SW1(config-vlan)#name Data_vlan_3

    LAB-SW1(config-vlan)#vlan 4

    LAB-SW1(config-vlan)#name Voice_vlan_4

    LAB-SW1(config-vlan)#vlan 5

    LAB-SW1(config-vlan)#name Wireless_vlan_5

    LAB-SW1(config-vlan)#exit

    LAB-SW1(config)#interface range gi1/0 - 3

    LAB-SW1(config-if-range)#switchport mode access

    LAB-SW1(config-if-range)#switchport access vlan 5

    LAB-SW1(config-if-range)#no shut

    LAB-SW1(config-if-range)#interface range gi2/0 - 3

    LAB-SW1(config-if-range)#switchport mode access

    LAB-SW1(config-if-range)#switchport access vlan 2

    LAB-SW1(config-if-range)#switchport voice vlan 4

    LAB-SW1(config-if-range)#no shut

    LAB-SW1(config-if-range)#end

    LAB-SW1#exit
```

```
# Task
```

6 Check the `LAB-SW1` console window for debugging information.

```
LAB-SW1#

*Jan 11 19:24:31.981: Telnet2: 1 1 251 1

*Jan 11 19:24:31.982: TCP2: Telnet sent WILL ECHO (1)

*Jan 11 19:24:31.982: Telnet2: 2 2 251 3

*Jan 11 19:24:31.984: TCP2: Telnet sent WILL SUPPRESS-GA (3)

*Jan 11 19:24:31.984: Telnet2: 80000 80000 253 24

*Jan 11 19:24:31.985: TCP2: Telnet sent DO TTY-TYPE (24)

*Jan 11 19:24:31.985: Telnet2: 10000000 10000000 253 31

*Jan 11 19:24:31.986: TCP2: Telnet sent DO WINDOW-SIZE (31)

*Jan 11 19:24:32.036: TCP2: Telnet received DONT ECHO (1)

*Jan 11 19:24:32.037: TCP2: Telnet sent WONT ECHO (1)

*Jan 11 19:24:32.042: TCP2: Telnet received DONT SUPPRESS-GA (3)

*Jan 11 19:24:32.043: TCP2: Telnet sent WONT SUPPRESS-GA (3)

LAB-SW1#

*Jan 11 19:24:32.048: TCP2: Telnet received WONT TTY-TYPE (24)

*Jan 11 19:24:32.050: TCP2: Telnet sent DONT TTY-TYPE (24)

*Jan 11 19:24:32.054: TCP2: Telnet received WONT WINDOW-SIZE (31)

*Jan 11 19:24:32.054: TCP2: Telnet sent DONT WINDOW-SIZE (31)

*Jan 11 19:24:32.186: TCP2: Telnet received DONT ECHO (1)

*Jan 11 19:24:32.187: TCP2: Telnet received DONT SUPPRESS-GA (3)

*Jan 11 19:24:32.187: TCP2: Telnet received WONT TTY-TYPE (24)

*Jan 11 19:24:32.189: TCP2: Telnet received WONT WINDOW-SIZE (31)

LAB-SW1#
```

```
#   Task
```

```
*Jan 11 19:24:40.915: %SYS-5-CONFIG_I: Configured from console by pyne-
tauto on vty0 (192.168.183.132)
```

```
7   Now check the VLANs configured and the switch ports on the LAB-SW1 switch. Use show
    vlan, show run, and show ip interface brief to check the switch configuration
    changes. The show vlan example is shown here:
```

```
LAB-SW1#show vlan

VLAN Name                             Status    Ports

---- -------------------------------- --------- -------------------------
-----

1    default                          active    Gi0/0, Gi0/1, Gi0/2, Gi0/3

                                                Gi3/0, Gi3/1, Gi3/2, Gi3/3

2    Data_vlan_2                      active    Gi2/0, Gi2/1, Gi2/2, Gi2/3

3    Data_vlan_3                      active

4    Voice_vlan_4                     active    Gi2/0, Gi2/1, Gi2/2, Gi2/3

5    Wireless_vlan_5                  active    Gi1/0, Gi1/1, Gi1/2, Gi1/3

1002 fddi-default                     act/unsup

[...omitted for brevity]

------- --------- ----------------- ------------------------------------
----
```

You have successfully added four VLANs and configured eight switch ports in their respective VLANs. Now let's check how we can use loops to add multiple random VLANs in labs 3 and 4.

Telnet Lab 3: Configure Random VLANs Using a for Loop

In the previous lab, we configured VLANs 2 to 5 with multiple lines of code; here, you are going to practice how to add VLANs with fewer lines of code using a `for` loop. For simplicity, we are only adding five random VLANs in this lab, but we are using a `for` loop, and you can save time and the number of code lines you have to write.

Follow the steps to configure random VLANs 101, 202, 303, 404, and 505 on `LAB-SW1` via Telnet. You can continue using the `ubuntu20s1` Linux server (192.168.183.132) and `LAB-SW1` (192.168.183.101) for this lab. Copy the

`7.2.2_add_vlans_single.py` file and rename it as `7.2.3_add_vlans_for_loop.py` for this lab. At the end of this lab, `LAB-SW1` should be configured as shown in Figure **13-4**.

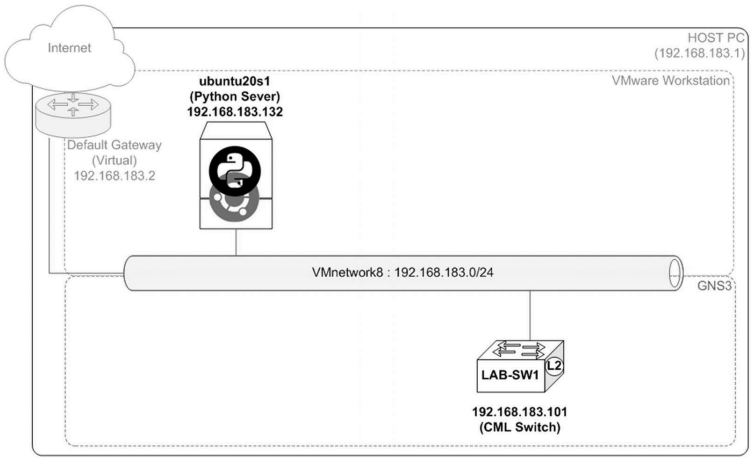


Figure 13-4. Telnet lab 3: devices in use

- Add VLANs 101, 202, 303, 404, and 505 with the following VLAN descriptions to `LAB-SW1`:
  - VLAN 101, `Data_vlan_101`
  - VLAN 202, `Data_vlan_202`
  - VLAN 303, `Voice_vlan_303`
  - VLAN 404, `Wireless_vlan_404`
  - VLAN 505, `Wireless_vlan_505`

In this lab, try to concentrate on how a `for` loop works and how it is used in this scenario. Loops are designed to carry out the same task repetitively, and now you are tapping into the true power of programming. Learning the Python concepts so you can apply them to your work is where you want to be. Good luck!

#	Task
1	First, move to the `telnet_labs` directory, use the `cp` command to copy `add_vlans_single.py`, and rename it `add_vlans_for_loop.py`. The commands are shown here:

```
pynetauto@ubuntu20s1:~$ cd telnet_labs
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ ls
```

```
7.2.1_add_lo_ospf1.py  7.2.2_add_vlans_single.py
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ cp add_vlans_single.py
add_vlans_for_loop.py
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ ls
```

```
add_lo_ospf1.py  add_vlans_single.py  add_vlans_for_loop.py
```

Task

```
pynetauto@ubuntu20s1:~/telnet_labs$ nano add_vlans_for_loop.py
```

2 Now, modify the script, so it looks like the following. Once again, press Ctrl+K to cut/copy the information, and press Ctrl+U to paste the copied information. Check your syntax and the whitespacing during this change. Every comma and space counts, especially the indentation or the four white spaces for code blocks.

```
GNU nano 4.8                    add_vlans_for_loop.py

#!/usr/bin/env python3 # This is called the shebang line; Python ignores
this line, but the Linux Operating System can read this line and knows
which application to run the .py file with.

import getpass

import telnetlib

HOST = "192.168.183.101"

user = input("Enter your username: ")

password = getpass.getpass()

tn = telnetlib.Telnet(HOST)

tn.read_until(b"Username: ")

tn.write(user.encode('ascii') + b"\n")

if password:

   tn.read_until(b"Password: ")

   tn.write(password.encode('ascii') + b"\n")

# Get into config mode

tn.write(b"conf t\n")

# Adds 5 vlans to the list with for loop

vlans = [101, 202, 303, 404, 505] # vlans to add in  a list

for i in vlans: # call (index) each item from list vlans

     command_1 = "vlan " + str(i) + "\n" # concatenate first command
```

```
# Task
```

```
    tn.write(command_1.encode('ascii')) # send command_1 with ASCII
encoding
```

```
    command_2 = "name PYTHON_VLAN_" + str(i) + "\n" # concatenate second
command
```

```
    tn.write(command_2.encode('ascii')) # send command_2 with ASCII
encoding
```

```
tn.write(b"end\n")
```

```
tn.write(b"exit\n")
```

```
print("exiting")
```

```
print(tn.read_all().decode('ascii'))
```

```
^G Get Help  ^O Write Out   ^W Where Is   ^K Cut Text     ^J Justify    ^C
Cur Pos
```

```
^X Exit      ^R Read File   ^\ Replace    ^U Paste Text   ^T To Spell   ^_
Go To Line
```

As you can see, the encoding and decoding are rather messy with `telnetlib`; that is because `telnetlib` uses pyNUT to communicate with network devices, and the pyNUT code is using string literals to format strings internally. PyNUT was written for Python 2, and `telnetlib` expects most inputs to be in bytes. In Python 2, the `str` (string) type was a byte string, but in Python 3, it is all Unicode. Fortunately, the encoding and decoding are not as complicated in SSH Python applications.

3 Run the script using the following Python command. You can use either command. We have added an alias of the user's `/.bashrc` to use either Python3 or Python to run the script .

```
pynetauto@ubuntu20s1:~/telnet_labs$ python add_vlans_for_loop.py
```

```
OR
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ python3 add_vlans_for_loop.py
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ python add_vlans_for_loop.py
```

```
Enter your username: pynetauto
```

```
Password:********
```

```
exiting
```

```
#    Task

     **************************************************************************

     * IOSv is strictly limited to use for evaluation, demonstration and IOS  *

     * education. IOSv is provided as-is and is not supported by Cisco's      *

     * Technical Advisory Center. Any use or disclosure, in whole or in part, *

     * of the IOSv Software or Documentation to any third party for any       *

     * purposes is expressly prohibited except as otherwise authorized by     *

     * Cisco in writing.                                                      *

     **************************************************************************

     LAB-SW1#conf t

     Enter configuration commands, one per line.  End with CNTL/Z.

     LAB-SW1(config)#vlan 101

     LAB-SW1(config-vlan)#name PYTHON_VLAN_101

     LAB-SW1(config-vlan)#vlan 202

     LAB-SW1(config-vlan)#name PYTHON_VLAN_202

     LAB-SW1(config-vlan)#vlan 303

     LAB-SW1(config-vlan)#name PYTHON_VLAN_303

     LAB-SW1(config-vlan)#vlan 404

     LAB-SW1(config-vlan)#name PYTHON_VLAN_404

     LAB-SW1(config-vlan)#vlan 505

     LAB-SW1(config-vlan)#name PYTHON_VLAN_505

     LAB-SW1(config-vlan)#end

     LAB-SW1#exit
```

4 Run the `show vlan` command on `LAB-SW1` to confirm newly configured VLANs. If you can see the new VLANs in the switch's vlan table, then your Python script used a for loop to add random VLANs to the switch.

```
#   Task

    LAB-SW1#show vlan

    VLAN Name                             Status    Ports
    ---- -------------------------------- --------- -------------------------
    -----
    1    default                          active    Gi0/0, Gi0/1, Gi0/2, Gi0/3
                                                    Gi3/0, Gi3/1, Gi3/2, Gi3/3
    2    Data_vlan_2                      active    Gi2/0, Gi2/1, Gi2/2, Gi2/3
    3    Data_vlan_3                      active
    4    Voice_vlan_4                     active    Gi2/0, Gi2/1, Gi2/2, Gi2/3
    5    Wireless_vlan_5                  active    Gi1/0, Gi1/1, Gi1/2, Gi1/3
    101  PYTHON_VLAN_101                  active
    202  PYTHON_VLAN_202                  active
    303  PYTHON_VLAN_303                  active
    404  PYTHON_VLAN_404                  active
    505  PYTHON_VLAN_505                  active
    [...omitted for brevity]
```

Telnet Lab 4: Configure Random VLANs Using a while Loop

As in the previous lab, this time we will create the same VLANs 101, 202, 303, 404, 505 on `lab-sw2` via Telnet. For this lab, you can continue working from the `ubuntu20s1` Linux server (192.168.183.132), but you also have to power on `lab-sw2` (192.168.183.102). Copy the `add_vlans_for_loop.py` file and create a new script called `add_vlans_while_loop.py`. At the end of this lab, `lab-sw2` should be configured with the same set of VLANs as `LAB-SW1`, as shown in Figure **13-5**.

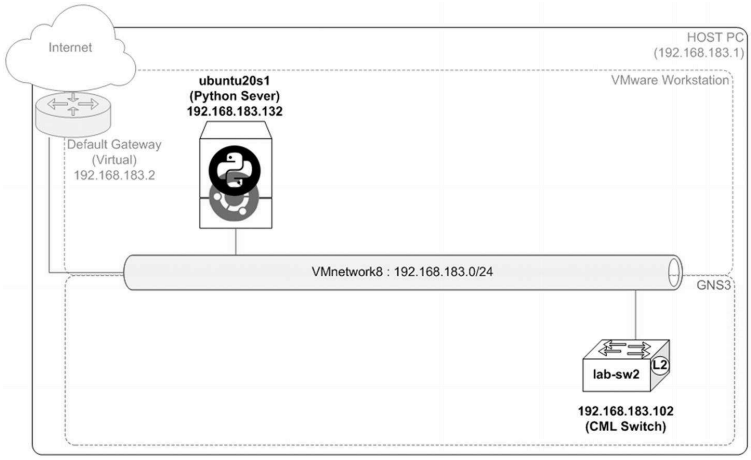


Figure 13-5. Telnet lab 4, devices in use

Add VLANs 101, 202, 303, 404 and 505 with the following VLAN descriptions to `lab-sw2`.

- VLAN 101, `Data_vlan_101`
- VLAN 202, `Data_vlan_202`
- VLAN 303, `Voice_vlan_303`
- VLAN 404, `Wireless_vlan_404`
- VLAN 505, `Wireless_vlan_505`

#	Task
1	Assuming that you are already working in the `telnet_labs` directory, use the `cp` command to copy `add_vlans_for_loop.py` and make `add_vlans_while_loop.py`. Follow the commands to create the new scrip shown here:

```
pynetauto@ubuntu20s1:~/telnet_labs$ ls

add_lo_ospf1.py      add_vlans_single.py       add_vlans_for_loop.py

pynetauto@ubuntu20s1:~/telnet_labs$ cp      add_vlans_for_loop.py   add_vlans_while_loop.py

pynetauto@ubuntu20s1:~/telnet_labs$ ls

add_lo_ospf1.py  add       vlans_single.py  add_     vlans_for_loop.py       add_vlans_while

pynetauto@ubuntu20s1:~/telnet_labs$ nano add_vlans_while_loop.py
```

2	Now modify your new script so it looks like the following. This script uses a `while` loop example to add the sa as the `for` loop example shown in Figure 13.3. Again, don't be so concerned about the actual login protocol yo ing here. Try to understand the working logic of the `while` loop used in this example script. See the embedded tion for more information .

```
GNU nano 4.8                    add_vlans_while_loop.py

#!/usr/bin/env python3

import getpass
```

```
# Task

import telnetlib

HOST = "192.168.183.102" # Update this to lab-sw2 switch IP

user = input("Enter your username: ")

password = getpass.getpass()

tn = telnetlib.Telnet(HOST)

tn.read_until(b"Username: ")

tn.write(user.encode('ascii') + b"\n")

if password:

  tn.read_until(b"Password: ")

  tn.write(password.encode('ascii') + b"\n")

# Get into to config mode

tn.write(b"conf t\n")

# Add 5 random vlans to vlans list and use  while loop to configure them to the switch

vlans = [101, 202, 303, 404, 505] # vlans to add in list

i = 0 # initial index value

while i < len(vlans): # while i is smaller than the length of vlans

  print(vlans[i]) # print vlans item

  command_1 = "vlan " + str(vlans[i]) + "\n" # concatenate first command

  tn.write(command_1.encode('ascii')) # send command_1 with ASCII encoding

  command_2 = "name PYTHON_VLAN_" + str(vlans[i]) + "\n" # concatenate second command

  tn.write(command_2.encode('ascii')) # send command_2 with ASCII encoding

  i +=1 # Same as i = i + 1

tn.write(b"end\n")

tn.write(b"exit\n")
```

```
# Task

print("exiting")
```

```
print(tn.read_all().decode('ascii'))
```

```
^G Get Help  ^O Write Out   ^W Where Is   ^K Cut Text     ^J Justify    ^C Cur Pos
^X Exit      ^R Read File   ^\ Replace    ^U Paste Text   ^T To Spell   ^_ Go To Line
```

3 After you have finished the `while` loop script, check the connection to `lab-sw2` (192.168.183.102) and run the using either of the commands shown here :

```
pynetauto@ubuntu20s1:~/telnet_labs$ python add_vlans_while_loop.py
```

```
OR
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ python3 add_vlans_while_loop.py
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ ping 192.168.183.102 -c 3
PING 192.168.183.102 (192.168.183.102) 56(84) bytes of data.
64 bytes from 192.168.183.102: icmp_seq=1 ttl=255 time=41.1 ms
64 bytes from 192.168.183.102: icmp_seq=2 ttl=255 time=27.3 ms
64 bytes from 192.168.183.102: icmp_seq=3 ttl=255 time=22.5 ms
--- 192.168.183.102 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 22.537/30.294/41.086/7.870 ms
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ python add_vlans_while_loop.py
Enter your username: pynetauto
Password:********
101
202
303
404
505
```

```
# Task

exiting

**************************************************************************

* IOSv is strictly limited to use for evaluation, demonstration and IOS  *

* education. IOSv is provided as-is and is not supported by Cisco's      *

* Technical Advisory Center. Any use or disclosure, in whole or in part, *

* of the IOSv Software or Documentation to any third party for any       *

* purposes is expressly prohibited except as otherwise authorized by     *

* Cisco in writing.                                                      *

**************************************************************************

lab-sw2#conf t

Enter configuration commands, one per line.  End with CNTL/Z.

lab-sw2(config)#vlan 101

lab-sw2(config-vlan)#name PYTHON_VLAN_101

lab-sw2(config-vlan)#vlan 202

lab-sw2(config-vlan)#name PYTHON_VLAN_202

lab-sw2(config-vlan)#vlan 303

lab-sw2(config-vlan)#name PYTHON_VLAN_303

lab-sw2(config-vlan)#vlan 404

lab-sw2(config-vlan)#name PYTHON_VLAN_404

lab-sw2(config-vlan)#vlan 505

lab-sw2(config-vlan)#name PYTHON_VLAN_505

lab-sw2(config-vlan)#end

lab-sw2#exit
```

4 Run the `show vlan` command from `lab-sw2` to check the newly configured VLANs. If you see the VLANs adde you have successfully configured random VLANs on a switch using a `while` loop.

```
#   Task

    lab-sw2#show vlan

    VLAN Name                             Status    Ports

    ---- -------------------------------- --------- -------------------------------

    1    default                          active    Gi0/0, Gi0/1, Gi0/2, Gi0/3

                                                    Gi1/0, Gi1/1, Gi1/2, Gi1/3

                          Gi2/0, Gi2/1, Gi2/2, Gi2/3

                          Gi3/0, Gi3/1, Gi3/2, Gi3/3

    101  PYTHON_VLAN_101                  active

    202  PYTHON_VLAN_202                  active

    303  PYTHON_VLAN_303                  active

    404  PYTHON_VLAN_404                  active

    505  PYTHON_VLAN_505                  active

    [...omitted for brevity]
```

Telnet Lab 5: Configure 100 VLANs Using the for ~ in range Loop Method

In this lab, you will create 100 VLANs using the `for ~ in range` loop method on both the `LAB-SW1` (192.168.183.101) and `lab-sw2` (192.168.183.102) switches. You will create the script from the same Python server, the `ubuntu20s1` Linux server (192.168.183.132). Copy the `add_vlans_while_loop.py` file and create a new script called `add_vlans_for_range.py`. At the end of this lab, both `LAB-SW1` and `lab-sw2` should be configured with VLANs 700 to 799. See Figure **13-6**.

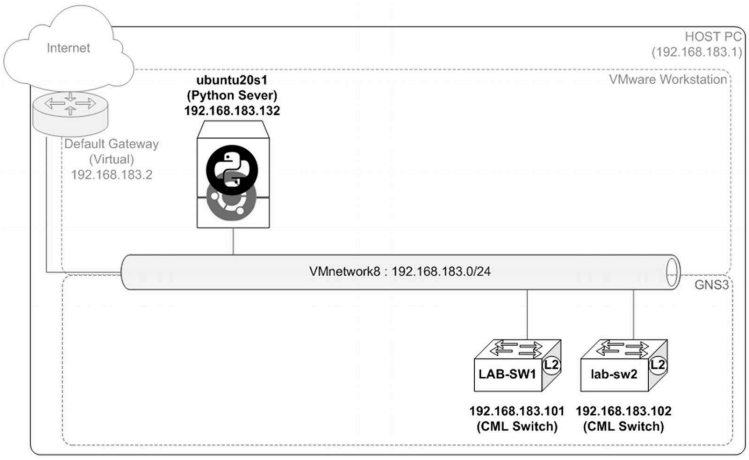


Figure 13-6. Telnet lab 5, devices in use

- Configure VLANs 700–799 with the following VLAN description:
  - VLANs 700–799 (i.e., `PYTHON_VLAN_700` to `PYTHON_VLAN_799`)

#	Task
1	Use PuTTY to SSH into `ubuntu20s1` (192.168.183.132) to copy the previous Telnet script and create the `7.2.5_add_100_vlans.py` file.

```
pynetauto@ubuntu20s1:~$ cd telnet_labs
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ ls
```

```
add_lo_ospf1.py      add_vlans_for_loop.py      add_vlans_single.py  add_vlans_while_loop.py
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ cp add_vlans_while_loop.py
add_100_vlans.py    pynetauto@ubuntu20s1:~/telnet_labs$ nano add_100_vlans.py
```

#	Task
2	This time you will be configuring 100 VLANs using a `for ~ range loop` on both switches using a single script. You have to be extremely careful with the leading whitespaces and the code blocks for this script. Be consistent with the whitespaces and the script's code blocks. Refer to the embedded explanation as you modify your script. Use `#` or `""" """` to add as many comments as you want. At the end of the modification, your Python script should look similar to the following code.

This script is available for download, but you should try to modify the script on Linux's nano text editor to get familiar with text editing on Linux .

```
GNU nano 4.8                        add_100_vlans.py
```

```
#!/usr/bin/env python3.8
```

```
import getpass
```

```
import telnetlib
```

```
# HOSTS is a list with IP addresses of two switches
```

```
#   Task

    HOSTS = ["192.168.183.101", "192.168.183.102"] # Create a list called HOSTS with two IPs

    user = input("Enter your username: ")

    password = getpass.getpass()

    # Use for loop to loop through the switch IPs

    # The rest of the script have been indented  to run under this block

    for HOST in HOSTS: # To loop through list, HOSTS

      print("SWITCH IP : " + HOST) # Marker to print out device IP

      tn = telnetlib.Telnet(HOST)

      tn.read_until(b"Username: ")

      tn.write(user.encode('ascii') + b"\n")

      if password:

         tn.read_until(b"Password: ")

         tn.write(password.encode('ascii') + b"\n")

      # Configure 100 VLANs with names using 'for ~ in range' loop # comment

      tn.write(b"conf t\n")

      # Use for n in range (starting vlan, ending vlan, (optional-stepping not used)) # comment

      for n in range (700, 800): # vlan range to add, 700 - 799, the last number is not counted

         tn.write(b"vlan " + str(n).encode('UTF-8') + b"\n") # Now part of vlan for loop

         tn.write(b"name PYTHON_VLAN_" + str(n).encode('UTF-8') + b"\n") # Now part of vlan for
    loop

      tn.write(b"end\n")

      tn.write(b"exit\n")

      print(tn.read_all().decode('ascii'))

    ^G Get Help  ^O Write Out   ^W Where Is   ^K Cut Text     ^J Justify    ^C Cur Pos

    ^X Exit      ^R Read File   ^\ Replace    ^U Paste Text   ^T To Spell   ^_ Go To Line
```

```
# Task
```

3 Your Python server needs good network connectivity to both switches from your Ubuntu Python server; check the switches' communication. Run the script only if your server can reach both IP addresses. If you have any communication issues, you will have to troubleshoot the issue before proceeding to the next step .

```
pynetauto@ubuntu20s1:~/telnet_labs$ ping 192.168.183.101 -c 3
PING 192.168.183.101 (192.168.183.101) 56(84) bytes of data.
64 bytes from 192.168.183.101: icmp_seq=1 ttl=255 time=5.15 ms
64 bytes from 192.168.183.101: icmp_seq=2 ttl=255 time=5.02 ms
64 bytes from 192.168.183.101: icmp_seq=3 ttl=255 time=5.42 ms
--- 192.168.183.101 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 5.019/5.208/5.419/0.145 ms
pynetauto@ubuntu20s1:~/telnet_labs$ ping 192.168.183.102 -c 3
PING 192.168.183.102 (192.168.183.102) 56(84) bytes of data.
64 bytes from 192.168.183.102: icmp_seq=1 ttl=255 time=9.91 ms
64 bytes from 192.168.183.102: icmp_seq=2 ttl=255 time=9.60 ms
64 bytes from 192.168.183.102: icmp_seq=3 ttl=255 time=9.55 ms
--- 192.168.183.102 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 9.549/9.726/9.909/0.154 ms
```

4 The network connection seems to be fine. Now, let's run the `add_100_vlans.py` script to add 100 VLANs on both switches. This script will Telnet into the first switch to add 100 VLANs and then Telnet into the second switch to add another 100 VLANs. This script may take a few minutes to complete, so be patient. Also, if your lab configuration is on an older computer like mine, consider reducing the number of `range` from 100 to 10 so you speed up the process .

```
pynetauto@ubuntu20s1:~/telnet_labs$ python add_100_vlans.py
Enter your username: pynetauto
Password:*******
```

```
# Task

SWITCH IP : 192.168.183.101

**************************************************************************

* IOSv is strictly limited to use for evaluation, demonstration and IOS  *

* education. IOSv is provided as-is and is not supported by Cisco's      *

* Technical Advisory Center. Any use or disclosure, in whole or in part, *

* of the IOSv Software or Documentation to any third party for any       *

* purposes is expressly prohibited except as otherwise authorized by     *

* Cisco in writing.                                                      *

**************************************************************************

LAB-SW1#conf t

Enter configuration commands, one per line.  End with CNTL/Z.

LAB-SW1(config)#vlan 700

LAB-SW1(config-vlan)#name PYTHON_VLAN_700

LAB-SW1(config-vlan)#vlan 701

[...omitted for brevity]

LAB-SW1(config-vlan)#vlan 799

LAB-SW1(config-vlan)#name PYTHON_VLAN_799

LAB-SW1(config-vlan)#end

LAB-SW1#exit

lab-sw2#

Enter configuration commands, one per line.  End with CNTL/Z.

[...omitted for brevity]
```

5 Give a brief moment for the script to complete its task and check both switches using `show vlan` to check the configuration changes. You should see newly configured VLANs, 700 to 799, on both switches .

```
Config message and check added vlans on LAB-SW1
```

```
# Task

LAB-SW1#

*Jan 11 21:38:00.558: %SYS-5-CONFIG_I: Configured from console by pynetauto on vty0
(192.168.183.132)

LAB-SW1#show vlan

[...omitted for brevity]

700  PYTHON_VLAN_700                 active

701  PYTHON_VLAN_701                 active

...

798  PYTHON_VLAN_798                 active

799  PYTHON_VLAN_799                 active

[...omitted for brevity]

Config message and check added vlans on lab-sw2

lab-sw2#

*Jan 11 21:47:09.432: %SYS-5-CONFIG_I: Configured from console by pynetauto on vty0
(192.168.183.132)

lab-sw2#show vlan

[...omitted for brevity]

700  PYTHON_VLAN_700                 active

701  PYTHON_VLAN_701                 active

...

798  PYTHON_VLAN_798                 active

799  PYTHON_VLAN_799                 active

[...omitted for brevity]
```

6 (Optional task) To remove (reverse) the 100 VLANs from the switches, you only need to modify two lines of code in the original script. The following example will copy the original script and rename it to `reverse_100_vlans.py`. Open this new file and modify two lines of code, as shown here, before running the script .

```
#   Task
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ cp add_100_vlans.py  reverse_100_vlans.py
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ nano reverse_100_vlans.py
```

```
Original code to update:...
```

```
 for n in range (700, 800):
```

```
    tn.write(b"vlan " + str(n).encode('UTF-8') + b"\n")
```

```
    tn.write(b"name PYTHON_VLAN_" + str(n).encode('UTF-8') + b"\n")
```

Updated reverse code:

```
...
```

```
 for n in range (700, 800):
```

```
    tn.write(b"no vlan " + str(n).encode('UTF-8') + b"\n")
```

```
    # tn.write(b"name PYTHON_VLAN_" + str(n).encode('UTF-8') + b"\n")
```

From your server, run **`python reverse_100_vlans.py`** to completely remove the vlans.

```
pynetauto@ubuntu20s1:~/telnet_labs$ python reverse_100_vlans.py
```

```
Enter your username: pynetauto
```

```
Password: ********
```

Telnet Lab 6: Add a Privilege 3 User on Multiple Devices Using IP Addresses from an External File

In this lab, you will configure a new user with limited privileges on all routers and switches in your network, so you will have to power on the devices shown in Figure 13-7. You have to create a separate file containing IP addresses so your script can read the IP addresses from the file and configure each device sequentially.

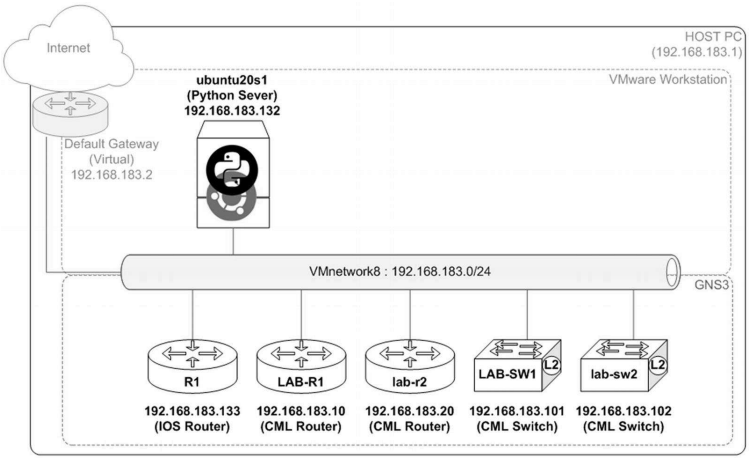


Figure 13-7. Telnet lab 6, devices in use

The new user should be given a junior network administrator privilege to run `show` commands. You will be assigning this new user a privilege 3 and allowing this user to view the network devices' running configuration and interface status. Also, change the file mode to the executable to run your script without typing `python` or `python3`.

At the end of this lab, you will have a local account created on each device, so the new user can run `show running-config view full`, `show ip interface brief`, and other `show` commands.

- *New username*: `junioradmin`
- *Password*: `cisco321`
- *Privilege level*: `3`
- *Command 1*: `privilege exec all level 3 show running-config`
- *Command 2*: `file privilege 3`

#	Task
1	On the `ubuntu20s1` Python server, create a text file containing all IP addresses of your network devices. Separ each device. To create and save this file, follow these instructions :

```
pynetauto@ubuntu20s1:~/telnet_labs$ touch ip_addresses.txt
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ nano ip_addresses.txt
```

```
GNU nano 4.8                    ip_addresses.txt
```

```
192.168.183.10
```

```
192.168.183.20
```

```
192.168.183.101
```

```
192.168.183.102
```

```
192.168.183.133
```

```
#   Task

    ^G Get Help  ^O Write Out    ^W Where Is   ^K Cut Text    ^J Justify   ^C Cur Pos

    ^X Exit      ^R Read File    ^\ Replace    ^U Paste Text  ^T To Spell  ^_ Go To Line
```

* Note: The `ip_addresses.txt` file must be in the same directory as the `add_junioradmin.py` file.

2 As with the previous steps, let's reuse the old script to create a new one. Copy the script from lab 5 and create

```
pynetauto@ubuntu20s1:~/telnet_labs$ cp add_100_vlans.py add_junioradmin.py

GNU nano 4.8                        add_junioradmin.py

#!/usr/bin/env python3

import getpass

import telnetlib

import time  # imports time module

user = input("Enter your username: ")

password = getpass.getpass()

# Open ip_addresses.txt to read IP addresses

file = open("ip_addresses.txt") # Open and read an external file

for ip in file: # loop through read information

  print("Now configuring : " + ip) # Task beginning statement

  HOST = (ip.strip()) # Strips any white spaces

  tn = telnetlib.Telnet(HOST) # Use read IP to log into a single device

  tn.read_until(b"Username: ")

  tn.write(user.encode('ascii') + b"\n")

  if password:

    tn.read_until(b"Password: ")

    tn.write(password.encode('ascii') + b"\n")

  time.sleep(1) # Adds 1 second pause for device to respond
```

```
# Task

  # Configure a new user with privilege 3, allow show running-config # comment

  tn.write(b"conf t\n") # Enter configuration mode

  tn.write(b"username junioradmin privilege 3 password cisco321\n") # Configure new pri 3 us

  tn.write(b"privilege exec all level 3 show running-config\n") # Allow show running-config

  print("Added a new privilege 3 user") # Task ending statement

  tn.write(b"end\n")

  tn.write(b"exit\n")

  print(tn.read_all().decode('ascii'))
```

```
^G Get Help  ^O Write Out    ^W Where Is   ^K Cut Text    ^J Justify   ^C Cur Pos

^X Exit      ^R Read File    ^\ Replace    ^U Paste Text  ^T To Spell  ^_ Go To Line
```

3 Before running the script, confirm that both the IP address and script files are in the same directory on your I

```
pynetauto@ubuntu20s1:~/telnet_labs$ ls

add_lo_ospf1.py        add_vlans_while_loop.py   ip_addresses.txt     add_vlans_single.py
```

4 From the `ubuntu20s1` server, check the connectivity to all the network devices in your network. We can instal Install it as shown here and ping all five devices in our topology .

```
pynetauto@ubuntu20s1:~/telnet_labs$ sudo apt install fping

pynetauto@ubuntu20s1:~/telnet_labs$ fping 192.168.183.10 192.168.183.20 192.168.183.101 192.

192.168.183.10 is alive

192.168.183.133 is alive

192.168.183.20 is alive

192.168.183.102 is alive

192.168.183.101 is alive
```

To read about how to use `fping`, please visit the following URL.

URL: https://www.2daygeek.com/how-to-use-ping-fping-gping-in-linux/

```
#    Task
```

We have been adding `#!/usr/bin/env python3.8` at the beginning, and as explained, this line is used so the
5 Python 3. To use this line, we first have to make our script executable. If you list the file we have just created a
the file .

```
pynetauto@ubuntu20s1:~/telnet_labs$ ls -l add_junior*

-rw-rw-r-- 1 pynetauto pynetauto 1298 Jan 12 01:16 add_junioradmin.py
```

For us to be able to run this script without using the `python` command, we can change the mode of this file us

```
pynetauto@ubuntu20s1:~/telnet_labs$ chmod +x ./ add_junioradmin.py

pynetauto@ubuntu20s1:~/telnet_labs$ ls -l add_junior*

-rwxrwxr-x 1 pynetauto pynetauto 1298 Jan 12 01:16 add_junioradmin.py
```

At this point, you can run your Python script using the `./add_junioradmin.py` command.

If you want to go one step further and want to run the script only using the script filename, you can add the fo
sessional while you are logged into your session. There are ways to set this permanently, but this is not covere
script only.

```
pynetauto@ubuntu20s1:~/telnet_labs$ PATH="$(pwd):$PATH"

pynetauto@ubuntu20s1:~/telnet_labs$ add_junioradmin.py

Enter your username: ^Z

[2]+  Stopped                 add_junioradmin.py
```

Press Ctrl+Z to exit if you want to run the script later; if not, continue to run the script as in step 6.

6 Let's run the script using the name of the file. The script will add the junior admin to all five devices, as shown

```
pynetauto@ubuntu20s1:~/telnet_labs$ add_junioradmin.py

Enter your username: pynetauto

Password:********

Now configuring : 192.168.183.10

Added a new privilege 3 user

**************************************************************************

* IOSv is strictly limited to use for evaluation, demonstration and IOS  *
```

```
# Task

    * education. IOSv is provided as-is and is not supported by Cisco's      *

    * Technical Advisory Center. Any use or disclosure, in whole or in part, *

    * of the IOSv Software or Documentation to any third party for any       *

    * purposes is expressly prohibited except as otherwise authorized by     *

    * Cisco in writing.                                                      *

    ***************************************************************************

    LAB-R1#conf t

    Enter configuration commands, one per line.  End with CNTL/Z.

    LAB-R1(config)#username junioradmin privilege 3 password cisco321

    LAB-R1(config)#privilege exec all level 3 show running-config

    LAB-R1(config)#file privilege 3

    LAB-R1(config)#end

    LAB-R1#exit

    Now configuring : 192.168.183.20

    Added a new privilege 3 user

    [...omitted for brevity]

    lab-sw2#conf t

    Enter configuration commands, one per line.  End with CNTL/Z.

    lab-sw2(config)#username junioradmin privilege 3 password cisco321

    lab-sw2(config)#privilege exec all level 3 show running-config

    lab-sw2(config)#end

    lab-sw2#exit

    Now configuring : 192.168.183.133

    Added a new privilege 3 user
```

```
# Task
```

```
R1#conf t
```

```
Enter configuration commands, one per line.  End with CNTL/Z.
```

```
R1(config)#username junioradmin privilege 3 password cisco321
```

```
R1(config)#privilege exec all level 3 show running-config
```

```
R1(config)#end
```

```
R1#exit
```

7 Log into each device and check the user configuration as well as the `privilege exec level 3` commands .

```
LAB-SW1#show run | in username junioradmin
```

```
username junioradmin privilege 3 password 0 cisco321
```

```
LAB-SW1#show run | in level 3
```

```
privilege exec all level 3 show running-config
```

```
privilege exec level 3 show
```

8 Now use PuTTY from your Windows host PC to log in with the `junioradmin` user and password `cisco321` and
mand. If you want to log into a device from the Ubuntu server, you can use Telnet 192.168.183.X, where X is th

The following example shows the login and commands run from the `LAB-SW1` switch and `lab-r2` router.

From your host PC, use PuTTY to log into `LAB-SW1` (192.168.183.101) via Telnet and log in as the `junioradmin` u

```
LAB-SW1#
```

```
[...omitted for brevity]
```

```
User Access Verification
```

```
Username: junioradmin
```

```
Password:cisco321
```

```
[...omitted for brevity]
```

```
LAB-SW1#show ip interface brief
```

```
Interface              IP-Address      OK? Method Status                Protocol
```

```
GigabitEthernet0/0     unassigned      YES unset  up                    up
```

```
#    Task

     GigabitEthernet0/1     unassigned      YES unset  up                    up

     GigabitEthernet0/2     unassigned      YES unset  up                    up

     GigabitEthernet0/3     unassigned      YES unset  down                  down

     [...omitted for brevity]
```

From your host PC, use PuTTY to log into `lab-r2` (192.168.183.20) via Telnet and log in as the `junioradmin` use

```
lab-r2#

[...omitted for brevity]

User Access Verification

Username: junioradmin

Password:cisco321

[...omitted for brevity]

lab-r2#show running-config view full

Building configuration...

Current configuration : 3625 bytes

!

! Last configuration change at 23:21:58 UTC Mon Jan 11 2021

!

version 15.6

service timestamps debug datetime msec

[...omitted for brevity]
```

You have successfully created a junior admin user account on multiple devices using IP addresses read from an external file. Note that the IP addresses do not have to be contiguous; they can be random IP addresses if you use a list or external files. What you have been learning from these Telnet labs can also be used on SSH labs with some minor modifications.

Telnet Lab 7: Taking Backups of running-config (or startup-config) to Local Server Storage

In Telnet lab 7, you will copy and modify the previous Telnet script to capture the current running configuration of each device in your network. The backed-up `running-config` files will be saved on the local drive. In this lab, you will be using the `datetime` module to get the timestamp and add the name of the files with this timestamp so you know when the backups were taken. You will require all routers and switches to be powered on, as in lab 6. See Figure 13-8.

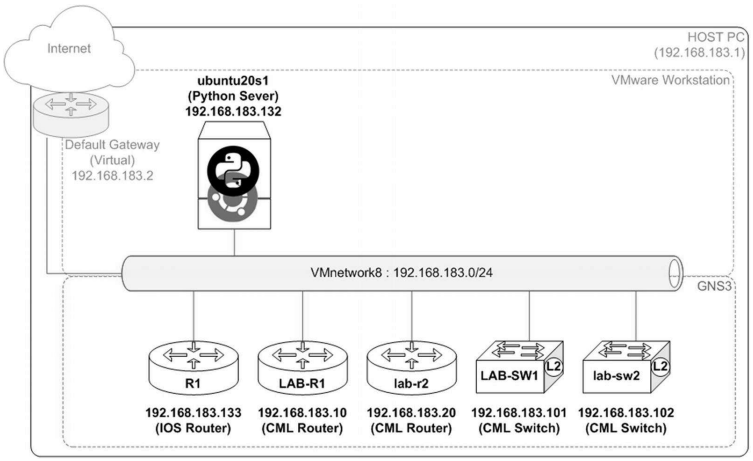


Figure 13-8. Telnet lab 7, devices in use

At the end of this lab, you will have the backups of each device's running configuration with the timestamp on your Python server's local storage.

#	Task
1	Once again, let's begin the lab by copying the script from the previous lab. The filename given here is `take_backups.py`, but you do not have to follow this naming convention; give a more meaningful name to this file so you can remember it easily.

```
pynetauto@ubuntu20s1:~/telnet_labs$ cp add_junior_user.py take_backups.py
```

```
pynetauto@ubuntu20s1:~/telnet_labs$ nano take_backups.py
```

2 Now, open the file in the nano (or vi) text editor and make the modifications highlighted in the following source code:

```
GNU nano 4.8                        take_backups.py
```

```
#!/usr/bin/env python3
```

```
import getpass
```

```
import telnetlib
```

```
from datetime import datetime # Import datetime module from datetime
library
```

```
saved_time = datetime.now().strftime("%Y%m%d_%H%M%S") # Change the \
```

```
# format of current time into a string
```

```
#    Task

    user = input("Enter your username: ") # Ask for username and password

    password = getpass.getpass()

    file = open("ip_addresses.txt") # Open ip_addresses.txt to read IP
    addresses

    # Telnets into Devices & runs show running-config and \

    # save it to a file with a timestamp

    for ip in file:

      print ("Getting running-config from " + (ip)) # First line print state-
    ment for admin

      HOST = ip.strip()

      tn = telnetlib.Telnet(HOST)

      tn.read_until(b"Username: ")

      tn.write(user.encode('ascii') + b"\n")

      if password:

        tn.read_until(b"Password: ")

        tn.write(password.encode('ascii') + b"\n")

      #Makes Term length to 0, run shows commands & reads all output, \

      # then saves files with time stamp # Comment

      tn.write(("terminal length 0\n").encode('ascii')) # Change terminal
    length to 0

      tn.write(("show clock\n").encode('ascii')) # Disply time

      tn.write(("show running-config\n").encode('ascii')) # Show running-
    configuration

      tn.write(("exit\n").encode('ascii')) # Exit session

      readoutput = tn.read_all() # Read output

      saveoutput = open(str(saved_time) + "_running_config_" + HOST, "wb") #
    saved_time is the time the file was saved, HOST is the IP address of the
```

```
#   Task
```

```
    device.
```

```
      saveoutput.write(readoutput) # Write the output to the file
```

```
    saveoutput.close # Save and close the file
```

```
    ^G Get Help  ^O Write Out    ^W Where Is   ^K Cut Text    ^J Justify   ^C
    Cur Pos
```

```
    ^X Exit      ^R Read File    ^\ Replace    ^U Paste Text  ^T To Spell  ^_
    Go To Line
```

3 Use the Linux `fping` command again to check the network connectivity on your network. If you have any nodes (network devices) not reachable, you must troubleshoot the connectivity issue first.

```
    pynetauto@ubuntu20s1:~/telnet_labs$ sudo apt install fping
```

```
    pynetauto@ubuntu20s1:~/telnet_labs$ fping 192.168.183.10 192.168.183.20
    192.168.183.101 192.168.183.102 192.168.183.133
```

```
    192.168.183.10 is alive
```

```
    192.168.183.133 is alive
```

```
    192.168.183.20 is alive
```

```
    192.168.183.102 is alive
```

```
    192.168.183.101 is alive
```

4 Now, run the script using the `./take_backup.py` command from your `ubuntu20s1` server. If you copied the script from the previous lab, the file attributes should be carried over, and your new script should be an executable file already. It is time to run the code for one last time in this chapter.

```
    pynetauto@ubuntu20s1:~/telnet_labs$ ls -l take*
```

```
    -rwxrwxr-x 1 pynetauto pynetauto 1574 Jan 12 10:50 take_backups.py
```

```
    pynetauto@ubuntu20s1:~/telnet_labs$ ./take_backups.py
```

```
    Enter your username: pynetauto
```

```
    Password:********
```

```
    Getting running-config from 192.168.183.10
```

```
# Task

    Getting running-config from 192.168.183.20

    Getting running-config from 192.168.183.101

    Getting running-config from 192.168.183.102

    Getting running-config from 192.168.183.133
```

4 After the script runs successfully, you can run the `ls -lh` Linux command to check your local directory's backed-up `running-config` files of your devices. The files should begin with the year, month, and day, followed by the time the backup was taken.

```
    pynetauto@ubuntu20s1:~/telnet_labs$ ls -lh 2021*

    -rw-rw-r-- 1 pynetauto pynetauto 4.3K Jan 12 10:53
    20210112_105318_running_config_192.168.183.10

    -rw-rw-r-- 1 pynetauto pynetauto 5.4K Jan 12 10:53
    20210112_105318_running_config_192.168.183.101

    -rw-rw-r-- 1 pynetauto pynetauto 4.8K Jan 12 10:54
    20210112_105318_running_config_192.168.183.102

    -rw-rw-r-- 1 pynetauto pynetauto 1.7K Jan 12 10:54
    20210112_105318_running_config_192.168.183.133

    -rw-rw-r-- 1 pynetauto pynetauto 4.5K Jan 12 10:53
    20210112_105318_running_config_192.168.183.20
```

5 Use the `cat` command to check if the correct information has been captured.

```
    pynetauto@ubuntu20s1:~/telnet_labs$ cat
    20210112_105318_running_config_192.168.183.10

    ******************************************************************************

    * IOSv is strictly limited to use for evaluation, demonstration and IOS  *

    * education. IOSv is provided as-is and is not supported by Cisco's      *

    * Technical Advisory Center. Any use or disclosure, in whole or in part, *

    * of the IOSv Software or Documentation to any third party for any       *

    * purposes is expressly prohibited except as otherwise authorized by     *

    * Cisco in writing.                                                      *
```

```
#    Task

     **************************************************************************

     LAB-R1#terminal length 0

     LAB-R1#show clock

     *08:01:50.307 UTC Tue Jan 12 2021

     LAB-R1#show running-config

     Building configuration...
     Current configuration : 3418 bytes

     !

     version 15.6

     service timestamps debug datetime msec

     service timestamps log datetime msec

     no service password-encryption

     !

     hostname LAB-R1

     !

     boot-start-marker

     boot-end-marker

     !

     !

     enable password cisco123

     !

     no aaa new-model

     [...omitted for brevity]
```

You have logged into each device and successfully backed up your routers' and switches' current running configurations to your local directory. Making the running configuration back up to an S/FTP server is even easier, but this lab has been reserved for SSH labs. At this point, try to

modify your script and run `write memory` or `copy running-config startup-config` to save all the configuration changes. Next, you will be attempting to do some SSH labs using the `paramiko` and `netmiko` libraries.

Summary

In this chapter, we focused on seven simple Telnet labs to leverage the knowledge we have gained from earlier chapters. You learned how to use the `fping` and `datetime` modules along the way. You also performed simple configuration changes interactively first and then by using Python scripts made changes using the power of loops and `range` commands. Also, you learned to read IP addresses from an external file and use the information in your script. In the final labs, you made configuration backups of all five network devices using Python `telnetlib` with the timestamp on each filename. In Chapter **14**, you will continue to explore Python network automation via SSH connections using the `paramiko` and `netmiko` SSH libraries.